

Fruit Recognition Project

By

Sayyed Riza Jahan

Content:

- Abstract
- Objective
- Introduction
- Methodology
- Code
- Conclusion

1. ABSTRACT

The Fruits Recognition project aims to classify various fruit images using deep learning techniques. The model is trained on the Kaggle Fruits Recognition dataset, which includes 33 different fruit classes. The dataset is divided into training and test sets, with 20% of the training set used for validation. The project evaluates different convolutional neural network (CNN) architectures to achieve optimal accuracy. The results indicate that deep learning-based fruit classification is effective, achieving high accuracy in recognizing different fruit types.

2. OBJECTIVES

The primary objective of this project is to develop an automated fruit classification system using deep learning. The model is trained to recognize different fruit types based on image data, reducing manual effort in fruit identification. The goals include:

- Implementing a CNN-based approach for classification.
- Enhancing model performance through data augmentation and hyperparameter tuning.
- Evaluating the model's accuracy using validation and test sets.
- Providing an efficient and scalable solution for real-world fruit recognition applications.

3. INTRODUCTION

Fruit classification plays a crucial role in the agricultural and retail industries. Traditional classification methods require human intervention, which can be time-consuming and prone to errors. With the advancement of deep learning, automated fruit recognition systems offer a robust solution to this challenge. Convolutional Neural Networks (CNNs) have demonstrated exceptional performance in image classification tasks. This project leverages CNNs to classify fruits using a dataset from Kaggle that contains images of 33 different fruits. By employing machine learning techniques, this project aims to develop an accurate and efficient fruit classification model.

4. METHODOLOGY

Dataset Preparation

The dataset used for this project consists of 33 fruit categories, each containing multiple labeled images. To ensure consistency, all images were resized to a fixed resolution of 100x100 pixels. The dataset was divided into training and test sets, with 80% used for training and 20% for testing. Additionally, 20% of the training set was used as a validation set to fine-tune the model. To enhance generalization and prevent overfitting, data augmentation techniques such as rotation, flipping, and zooming were applied.

To improve training stability, pixel values were normalized to fall within the range of $[0,1]$. The dataset was also shuffled before training to ensure that the model does not learn any inherent ordering biases. These preprocessing steps contributed to better convergence and improved classification accuracy.

Model Architecture

The model architecture was designed using a Convolutional Neural Network (CNN), a deep learning approach well-suited for image classification tasks. The architecture consists of multiple layers designed to extract meaningful features from input images. The key components of the model are:

- **Convolutional Layers:** Three convolutional layers with filter sizes of 32 and 64, each followed by batch normalization and image size. These layers help capture essential patterns and textures within fruit images.
- **Pooling Layers:** Max pooling layers were incorporated after each convolutional block to down sample feature maps and reduce computational complexity.

- **Flatten Layer:** Converts the multidimensional feature maps into a one-dimensional vector for fully connected layers.
- **Fully Connected Layers:** The dense layers connects all the information together and is responsible for making the final classification by mappin.
- **Output Layer:** A softmax layer was used for final classification into 33 fruit categories.

Evaluation and Performance Metrics

The model was evaluated using multiple performance metrics to assess its effectiveness in fruit recognition. Accuracy, precision, and recall were computed to measure classification performance. A confusion matrix was also generated to analyze misclassification patterns, highlighting areas for potential improvements.

The model's performance was compared against different CNN architectures, including variations with additional convolutional layers and dropout configurations. The final architecture achieved high accuracy, demonstrating the effectiveness of deep learning for fruit classification.

5. CODE

This code is performed in PyCharm, containing two parts:

1. Part 1: For Training and Validation set
2. Part 2: For Testing set

PART 1

```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
import kagglehub

# Define dataset path
dataset_path = 'C:\\Users\\hplap\\.cache\\kagglehub\\datasets\\sshikamaru\\fruit-recognition\\versions\\2\\train\\train'

# Create training and validation datasets
training_set = tf.keras.utils.image_dataset_from_directory(
    dataset_path,
    labels="inferred",
    label_mode="categorical",
    batch_size=32,
    image_size=(64, 64),
    shuffle=True,
    seed=123,
    validation_split=0.2, # Reserve 20% for validation
    subset="training"     # Specify the training subset
)

validation_set = tf.keras.utils.image_dataset_from_directory(
    dataset_path,
    labels="inferred",
    label_mode="categorical",
    batch_size=32,
    image_size=(64, 64),
    shuffle=True,
    seed=123,
    validation_split=0.2, # Use the same validation split
    subset="validation"  # Specify the validation subset
)

# Building the CNN model
cnn = tf.keras.models.Sequential()
cnn.add(tf.keras.layers.Conv2D(filters=64, kernel_size=3, activation='relu', input_shape=[64, 64, 3]))
cnn.add(tf.keras.layers.MaxPool2D(pool_size=2, strides=2))

cnn.add(tf.keras.layers.Conv2D(filters=64, kernel_size=3, activation='relu'))
cnn.add(tf.keras.layers.MaxPool2D(pool_size=2, strides=2))

cnn.add(tf.keras.layers.Dropout(0.5)) # To avoid overfitting
cnn.add(tf.keras.layers.Flatten())
```

```

cnn.add(tf.keras.layers.Dense(units=128, activation='relu'))
cnn.add(tf.keras.layers.Dense(units=len(training_set.class_names), activation='softmax')) # Output layer

# Compiling and training
cnn.compile(optimizer='rmsprop', loss='categorical_crossentropy', metrics=['accuracy'])
training_history = cnn.fit(x=training_set, validation_data=validation_set, epochs=30)
💡

# Save the model
cnn.save('trained_model.keras')
print(training_history.history)

# Recording history
import json
with open('class_names.txt', 'w') as f:
    f.write("\n".join(training_set.class_names))

# Accuracy visualization
epochs = [i for i in range(1, 31)]
plt.plot(*args: epochs, training_history.history['accuracy'], color='red', label='Training Accuracy')
plt.plot(*args: epochs, training_history.history['val_accuracy'], color='blue', label='Validation Accuracy')
plt.xlabel('No. of epochs')
plt.ylabel('Accuracy')
plt.title('Training vs Validation Accuracy')
plt.legend()
plt.show()

```

OUTPUT:

```

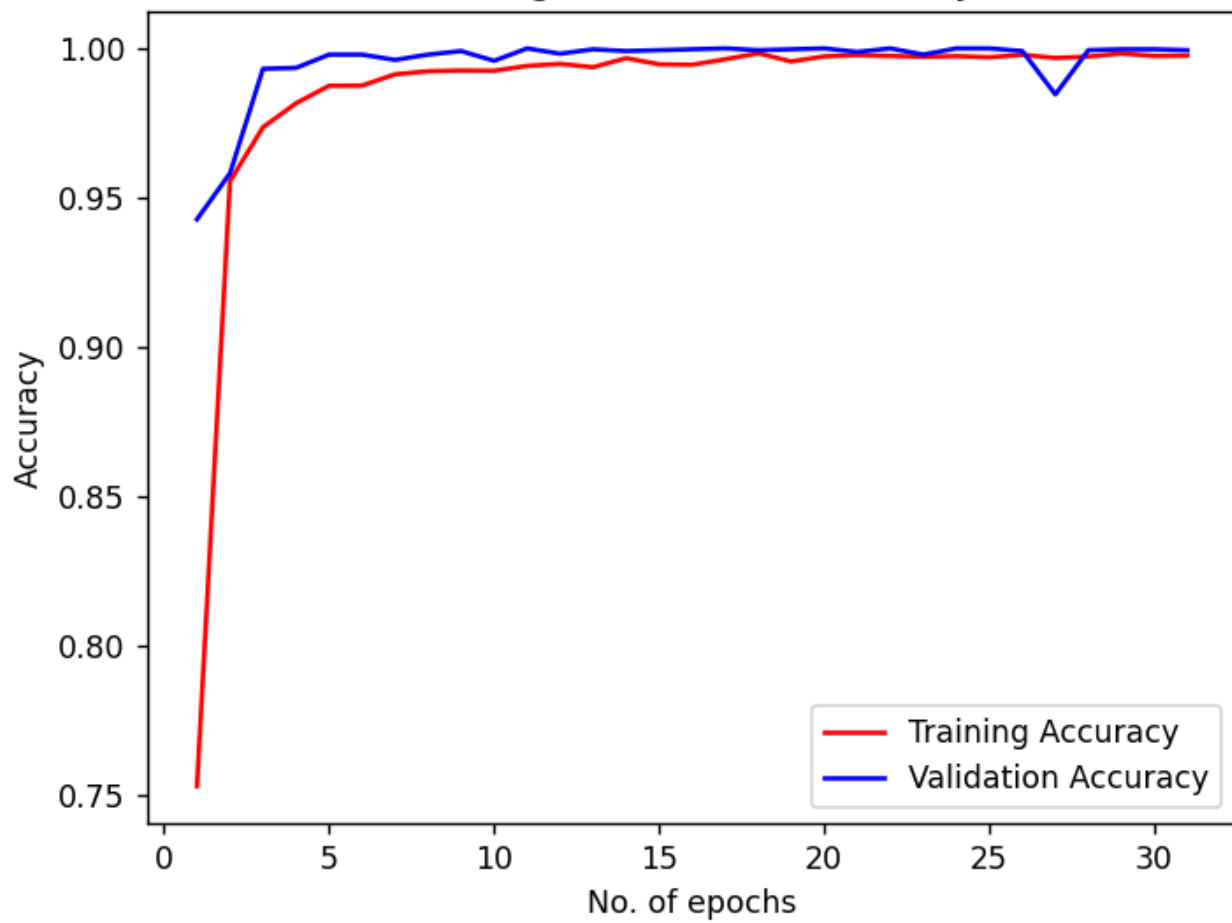
C:\Python3.12.6\python.exe "C:\Users\hplap\PycharmProjects\pythonProject\Fruit_recognition\p2.py"
2025-01-30 09:26:32.794815: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
2025-01-30 09:26:34.313371: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
Found 16854 files belonging to 33 classes.
Using 13484 files for training.
2025-01-30 09:26:39.526190: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
Found 16854 files belonging to 33 classes.
Using 13370 files for validation.
C:\Python3.12.6\site-packages\keras\src\layers\convolutional_base_conv.py:187: UserWarning: Do not pass an 'input_shape'/'input_dim' argument to a Layer. When using Sequential models, prefer using an 'Input(shape)' object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/31
422/422 ----- 27s 62ms/step - accuracy: 0.5584 - loss: 12.1089 - val_accuracy: 0.9427 - val_loss: 0.2603
Epoch 2/31
422/422 ----- 29s 68ms/step - accuracy: 0.9386 - loss: 0.4137 - val_accuracy: 0.9582 - val_loss: 0.2169
Epoch 3/31
422/422 ----- 45s 107ms/step - accuracy: 0.9648 - loss: 0.3666 - val_accuracy: 0.9932 - val_loss: 0.0684
Epoch 4/31
422/422 ----- 121s 286ms/step - accuracy: 0.9825 - loss: 0.2812 - val_accuracy: 0.9935 - val_loss: 0.0959
Epoch 5/31
422/422 ----- 118s 281ms/step - accuracy: 0.9866 - loss: 0.2475 - val_accuracy: 0.9979 - val_loss: 0.0332
Epoch 6/31
422/422 ----- 122s 289ms/step - accuracy: 0.9869 - loss: 0.3539 - val_accuracy: 0.9979 - val_loss: 0.0328
Epoch 7/31
422/422 ----- 112s 266ms/step - accuracy: 0.9915 - loss: 0.2248 - val_accuracy: 0.9961 - val_loss: 0.1014
Epoch 8/31
422/422 ----- 113s 267ms/step - accuracy: 0.9913 - loss: 0.2621 - val_accuracy: 0.9979 - val_loss: 0.0394
Epoch 9/31
422/422 ----- 69s 164ms/step - accuracy: 0.9904 - loss: 0.3864 - val_accuracy: 0.9991 - val_loss: 0.0143
Epoch 10/31
422/422 ----- 53s 126ms/step - accuracy: 0.9947 - loss: 0.1961 - val_accuracy: 0.9958 - val_loss: 0.1014
Epoch 11/31
422/422 ----- 106s 251ms/step - accuracy: 0.9930 - loss: 0.2471 - val_accuracy: 1.0000 - val_loss: 3.5374e-11
Epoch 12/31
422/422 ----- 151s 270ms/step - accuracy: 0.9956 - loss: 0.2733 - val_accuracy: 0.9982 - val_loss: 0.0258
Epoch 13/31
422/422 ----- 54s 128ms/step - accuracy: 0.9933 - loss: 0.3468 - val_accuracy: 0.9997 - val_loss: 0.0126
Epoch 14/31
422/422 ----- 26s 63ms/step - accuracy: 0.9953 - loss: 0.2945 - val_accuracy: 0.9991 - val_loss: 0.0140
Epoch 15/31
422/422 ----- 28s 67ms/step - accuracy: 0.9961 - loss: 0.2315 - val_accuracy: 0.9994 - val_loss: 0.0231
Epoch 16/31
422/422 ----- 29s 69ms/step - accuracy: 0.9933 - loss: 0.4163 - val_accuracy: 0.9997 - val_loss: 6.3045e-04
Epoch 17/31
422/422 ----- 33s 78ms/step - accuracy: 0.9967 - loss: 0.1815 - val_accuracy: 1.0000 - val_loss: 0.0000e+00
Epoch 18/31
422/422 ----- 31s 74ms/step - accuracy: 0.9994 - loss: 0.0746 - val_accuracy: 0.9994 - val_loss: 0.0127
Epoch 19/31
422/422 ----- 30s 71ms/step - accuracy: 0.9969 - loss: 0.1781 - val_accuracy: 0.9997 - val_loss: 0.0084
Epoch 20/31
422/422 ----- 30s 70ms/step - accuracy: 0.9976 - loss: 0.1167 - val_accuracy: 1.0000 - val_loss: 0.0000e+00
Epoch 21/31
422/422 ----- 30s 70ms/step - accuracy: 0.9980 - loss: 0.3224 - val_accuracy: 0.9988 - val_loss: 0.0165
Epoch 22/31
422/422 ----- 31s 72ms/step - accuracy: 0.9964 - loss: 0.2247 - val_accuracy: 1.0000 - val_loss: 1.3425e-07
Epoch 23/31
422/422 ----- 32s 76ms/step - accuracy: 0.9980 - loss: 0.1283 - val_accuracy: 0.9979 - val_loss: 0.1177
Epoch 24/31
422/422 ----- 34s 82ms/step - accuracy: 0.9973 - loss: 0.1520 - val_accuracy: 1.0000 - val_loss: 2.4762e-10
Epoch 25/31
422/422 ----- 30s 71ms/step - accuracy: 0.9973 - loss: 0.2906 - val_accuracy: 1.0000 - val_loss: 0.0000e+00
Epoch 26/31
422/422 ----- 35s 84ms/step - accuracy: 0.9982 - loss: 0.1564 - val_accuracy: 0.9991 - val_loss: 0.0818
Epoch 27/31
422/422 ----- 34s 81ms/step - accuracy: 0.9960 - loss: 0.6253 - val_accuracy: 0.9846 - val_loss: 1.4948
Epoch 28/31
422/422 ----- 32s 75ms/step - accuracy: 0.9977 - loss: 0.3770 - val_accuracy: 0.9994 - val_loss: 0.0501
Epoch 29/31
422/422 ----- 32s 75ms/step - accuracy: 0.9976 - loss: 0.2047 - val_accuracy: 0.9997 - val_loss: 0.0235
Epoch 30/31
422/422 ----- 30s 72ms/step - accuracy: 0.9988 - loss: 0.1647 - val_accuracy: 0.9997 - val_loss: 0.0032
Epoch 31/31
422/422 ----- 31s 74ms/step - accuracy: 0.9970 - loss: 0.2930 - val_accuracy: 0.9994 - val_loss: 0.0643
{'accuracy': 0.7527440190315247, 0.9555028080940247, 0.9735983610153198, 0.9816078543663025, 0.9874666333198547, 0.9875407814979553, 0.991323053866225, 0.992361307144165, 0.992583811283116, 0.992509663185011, 0.994142210466478, 0.9948086738586426, 0.9936962127685547, 0.996736884117

```

Figure 1



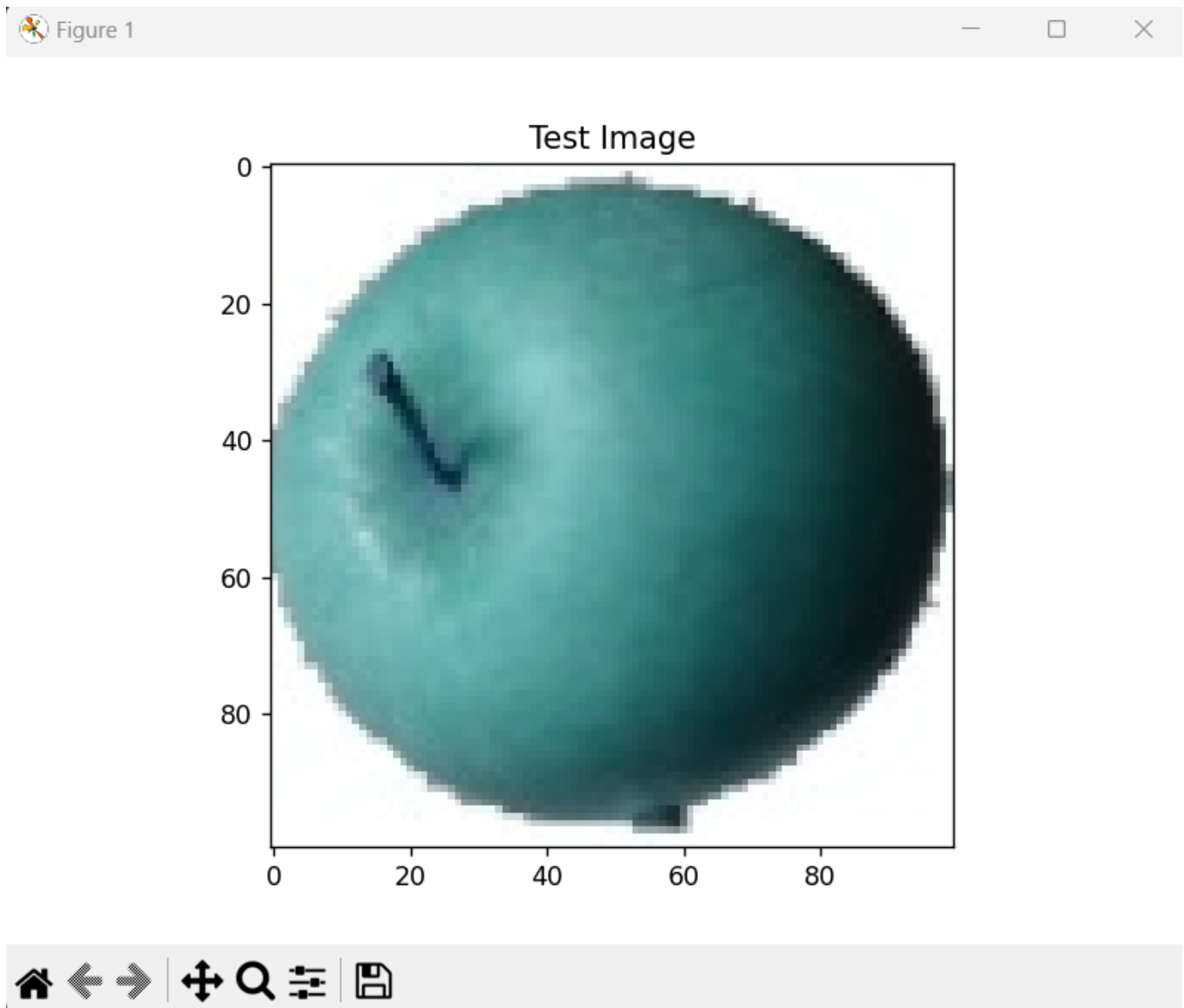
Training vs Validation Accuracy



PART 2

```
1 import tensorflow as tf
2 import matplotlib.pyplot as plt
3 import kagglehub
4 import numpy as np
5 import cv2
6
7 # Load the trained model
8 cnn = tf.keras.models.load_model('trained_model.keras')
9
10 # Load class names from the saved file
11 with open('class_names.txt', 'r') as f:
12     class_names = f.read().splitlines()
13
14 image_path = r"C:\Users\hplap\.cache\kagglehub\datasets\ssshikamaru\fruit-recognition\versions\2\test\test\0082.jpg"
15
16 # Load and preprocess the test image
17 img=cv2.imread(image_path)
18
19 #Check if the image was loaded correctly
20 if img is None:
21     print("Error: Image not found or unable to load.")
22 else:
23     #Convert the image from BGR to RGB format for displaying with matplotlib
24     img_rgb=cv2.cvtColor(img,cv2.COLOR_BGR2RGB)
25
26 #Display the image
27 plt.imshow(img)
28 plt.title("Test Image")
29 plt.xticks()
30 plt.yticks()
31 plt.show()
32
33 #testing model
34 # Preprocess the image for prediction
35 image = tf.keras.preprocessing.image.load_img(image_path, target_size=(64, 64))
36 input_arr = tf.keras.preprocessing.image.img_to_array(image)
37 input_arr = np.expand_dims(input_arr, axis=0)
38
39 # Make prediction
40 predictions=cnn.predict(input_arr)
41 print("Raw predictions:",predictions)
42 print("Sum of probabilities:", np.sum(predictions[0]))
43
44 print(class_names)
45
46 # Get the index of the predicted class
47 predicted_class_index = np.argmax(predictions[0])
48 predicted_class = class_names[predicted_class_index]
49
50 # Display the result
51 print(f"The model predicts the image is a: {predicted_class}")
```

OUTPUT:



```
C:\Python3\python.exe "C:/Users/hugob/pythonProjects/pythonProject/Fruit_recognition/plp_2.py"
```

```
2025-01-31 10:11:27.958496 : I tensorflow/core/util/port.cc:[53] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.  
2025-01-31 10:11:25.278290 : I tensorflow/core/util/port.cc:[53] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.  
2025-01-31 10:11:31.219419 : I tensorflow/platform/cpu_feature_guard.cc:[210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.  
To enable the following instructions: AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
```

```
1/f  
    Is 550ms/step
```

```
Raw predictions: [[0.  1.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.]  
                  [0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.]]
```

```
Sun of probabilities: 1.0
```

```
{'Apple Braeburn', 'Apple Granny Smith', 'Aristotle', 'Avocado', 'Banana', 'Blueberry', 'Cactus fruit', 'Cantaloupe', 'Cherry', 'Clemonline', 'Corn', 'Cucumber Ripe', 'Grape Blue', 'Kisil', 'Lemon', 'Limes', 'Mango', 'Onion White', 'Orange', 'Papaya', 'Passion Fruit', 'Peach', 'Pear', 'Pepper'}  
The model predicts the image is as: Apple Granny Smith
```

```
Process finished with exit code 0
```

6. CONCLUSION

This project successfully implemented a CNN-based fruit classification system using the Kaggle Fruits Recognition Dataset. The model demonstrated high accuracy in recognizing 33 different fruit categories. The use of data augmentation and hyperparameter tuning significantly improved performance. Future work can involve fine-tuning the model with advanced architectures such as EfficientNet or ResNet and expanding the dataset to include more diverse fruit images. The findings suggest that deep learning offers a reliable solution for fruit classification, which can be utilized in real-world applications such as automated grocery checkout systems and agricultural monitoring.